Argustack Architecture Guide

Version 0.1.12 | March 2026

Open-source CLI tool that downloads project data from Jira, Git, GitHub, and external databases into local PostgreSQL, then gives AI direct access via 20 MCP tools.

All data stays on your machine. Nothing leaves localhost.

Visit the project on GitHub or install from npm.

> **Note:** This documentation is actively maintained alongside the codebase. While we verify accuracy with each release, minor discrepancies may exist as features evolve. For the latest information, refer to the GitHub repository. Found an issue? Open a ticket.

Table of Contents

1. The Problem
2. Architecture Overview
3. Directory Structure
4. Data Flow
5. Cross-Reference System
6. MCP Tools (20)
7. Estimate Tool
8. Database Schema
9. Testing Strategy
10. Extending Argustack
11. Tech Stack

1. The Problem

Every project has multiple sources of truth that don't talk to each other:

Source	What it knows	What it doesn't know
Jira	What was planned, who was assigned	Whether code was actually written
Git	What code changed, by whom, when	Why it was written, what ticket it's for
GitHub	Who reviewed, who approved	The original business requirement
Database	What exists in production right now	How it got there

A project manager asks: "Was ticket PROJ-123 implemented as described?" To answer this, you'd need to open Jira, then search Git for commits mentioning PROJ-123, then check GitHub for the PR, then read the review comments. Four tools, manual cross-referencing.

Argustack solves this. It downloads everything into one database, automatically links commits and PRs to Jira issues, and gives AI direct SQL access to answer questions like these in seconds.

2. Architecture Overview

Hexagonal Architecture (Ports & Adapters)

Traditional layered architecture creates tight coupling. Adding a new data source (say, Linear instead of Jira) means touching every layer.

Hexagonal Architecture solves this with **ports and adapters**:

- **Ports** — interfaces that define what the system needs
- **Adapters** — concrete implementations that plug into ports

Adding Linear support = writing one new adapter. Everything else stays untouched.

The Two Sides

Driving adapters — how users interact with the system:

- `cli/` — Commander.js commands (`argustack sync`, `argustack init`)
- `mcp/` — MCP server for Claude Desktop and Claude Code

Driven adapters — external systems the app depends on:

- `adapters/jira/` — Jira REST API
- `adapters/csv/` — Jira CSV export parser
- `adapters/git/` — local Git repos via es-git (libgit2)
- `adapters/github/` — GitHub REST API via Octokit
- `adapters/db/` — external databases via Knex (PostgreSQL, MySQL, MSSQL, SQLite, Oracle)
- `adapters/postgres/` — PostgreSQL storage
- `adapters/openai/` — embeddings for semantic search

The Dependency Rule

> Dependencies always point inward. Core knows nothing about the outside world.

```
cli/, mcp/  →  use-cases/  →  core/ports  ←  adapters/
(driving)      (logic)        (contracts)     (driven)
```

- **core/** — zero external dependencies, only pure TypeScript types and interfaces
- **use-cases/** — depends only on core/ interfaces, doesn't know about Jira or PostgreSQL
- **adapters/** — implement core/ interfaces, each independently replaceable
- **cli/** — composition root, creates adapters, injects them into use cases

3. Directory Structure

```
src/
├── core/                       CORE: types + interfaces
│   ├── types/
│   │   ├── issue.ts               Issue, Comment, Changelog, Worklog, Link, HybridSearch
│   │   ├── git.ts                 Commit, CommitFile, CommitIssueRef, CommitBatch
│   │   ├── github.ts              PullRequest, Review, PullRequestFile, GitHubBatch
│   │   ├── project.ts             Project
│   │   ├── database.ts            DbEngine, DbTable, DbColumn, DbForeignKey, DbIndex
│   │   └── config.ts              WorkspaceConfig, SourceConfig, SourceType
│   └── ports/
│       ├── source-provider.ts     ISourceProvider
│       ├── git-provider.ts        IGitProvider (extends ISourceProvider)
│       ├── github-provider.ts     IGitHubProvider (extends ISourceProvider)
│       ├── db-provider.ts         IDbProvider (connect, introspect, query)
│       ├── embedding-provider.ts  IEmbeddingProvider
│       └── storage.ts             IStorage
│
├── use-cases/                  BUSINESS LOGIC
│   ├── pull.ts                    PullUseCase: Jira → PostgreSQL
│   ├── pull-git.ts                PullGitUseCase: Git → PostgreSQL
│   ├── pull-github.ts             PullGitHubUseCase: GitHub → PostgreSQL
│   ├── pull-db.ts                 PullDbUseCase: External DB schema → PostgreSQL
│   └── embed.ts                   EmbedUseCase: issues → OpenAI → pgvector
│
├── adapters/                   DRIVEN ADAPTERS
│   ├── jira/                      Jira REST API (client, mapper, provider)
│   ├── csv/                       Jira CSV import (parser, mapper, provider)
│   ├── git/                       Local Git repos — es-git (mapper, provider)
│   ├── github/                    GitHub REST API — Octokit (client, mapper, provider)
│   ├── db/                        External databases — Knex (client, validator, mapper,
│   ├── postgres/                  PostgreSQL storage (connection, schema, storage)
│   └── openai/                    OpenAI embeddings (embedding-provider)
│
├── mcp/                        DRIVING ADAPTER: Claude MCP
│   ├── server.ts                  Orchestrator
│   ├── types.ts                   SQL row interfaces
│   ├── helpers.ts                 Shared utilities
│   └── tools/                     20 tools across 6 modules
│       ├── workspace.ts
│       ├── query.ts
│       ├── issue.ts
│       ├── search.ts
│       ├── estimate.ts
│       └── database.ts
│
├── cli/                        DRIVING ADAPTER: Commander.js
│   ├── index.ts                   Command registration
│   ├── init/                      argustack init (interactive setup)
│   │   ├── setup-jira.ts             Jira source setup
│   │   ├── setup-git.ts             Git source setup
│   │   ├── setup-github.ts          GitHub source setup
│   │   ├── setup-csv.ts             CSV source setup
```

```
│   │   ├── setup-db.ts              DB source setup
│   │   ├── generators.ts            File generators (docker-compose, .env, .mcp.json)
│   │   └── types.ts                 Init types
│   ├── sync.ts                    argustack sync
│   ├── embed.ts                   argustack embed
│   ├── sources.ts                 argustack sources
│   ├── status.ts                  argustack status
│   └── mcp-install.ts             argustack mcp install
│
└── workspace/                  Config + workspace resolver
```

4. Data Flow

Sync: How Data Gets In

When you run `argustack sync jira`:

Step 1 — CLI creates the wiring:

```
sync.ts  →  JiraProvider(credentials)
         →  PostgresStorage(connectionString)
         →  PullUseCase(provider, storage)
```

Step 2 — Use case orchestrates:

```
PullUseCase.execute()
  → provider.pullIssues()     AsyncGenerator yields IssueBatch
  → storage.saveBatch()       UPSERT into PostgreSQL
  → repeat until exhausted
```

Step 3 — AsyncGenerator streams data:

Jira API returns 100 issues per page. Provider maps raw JSON to core types, yields a batch, storage saves it, moves to next. Memory stays flat regardless of dataset size — 10K or 100K issues.

Query: How Data Gets Out

When Claude calls an MCP tool:

```
Claude  →  MCP server  →  PostgreSQL  →  results  →  Claude
```

MCP tools query the database directly with SQL. No use cases involved — MCP is a driving adapter that reads storage directly. This is intentional: query tools are read-only and don't need business logic orchestration.

5. Cross-Reference System

The killer feature: automatic linking between sources.

When a commit message contains `PROJ-123`, Argustack extracts the issue key and creates a `commit_issue_refs` record. Same for PR titles and bodies → `pr_issue_refs`.

What This Enables

Question	MCP Tool
What code was written for PROJ-123?	`issue_commits`
Was PROJ-123 reviewed? By whom?	`issue_prs`
Full timeline from ticket creation to merge	`issue_timeline`
Which tickets were part of this release?	`query_releases` → `query_prs`

issue_timeline — The Full Picture

Combines all three sources into one chronological view for a single issue:

- Jira changelog events (status changes, reassignments)
- Git commits (code changes, file diffs)
- GitHub PRs (reviews, approvals, comments)

One tool call = complete picture of how a ticket went from idea to production.

6. MCP Tools (20)

After sync, Claude queries your data through these tools:

Jira

Tool	Purpose
`query_issues`	Full-text search, filters by status/type/assignee, raw SQL
`get_issue`	Full details: description, comments, changelogs, custom fields
`issue_stats`	Aggregates by status, type, assignee, project
`pull_jira`	Sync latest data from Jira (incremental)
`list_projects`	List available Jira projects

Git

Tool	Purpose
`query_commits`	Search by text, author, date, file path, raw SQL
`issue_commits`	All commits mentioning a Jira issue key
`commit_stats`	Top authors, most changed files, activity by date

GitHub

Tool	Purpose
`query_prs`	Search by text, state, author, base branch, raw SQL
`issue_prs`	All PRs mentioning a Jira issue key (with reviews)
`query_releases`	List releases with full-text search

Cross-Source

Tool	Purpose
`issue_timeline`	Chronological timeline: changelogs + commits + PRs
`hybrid_search`	Combined text + semantic search using RRF
`estimate`	Predict task duration per developer

Database

Tool	Purpose
`db_schema`	Browse external database schema (tables, columns, FKs, indexes)
`db_query`	Execute read-only SQL against the external database
`db_stats`	External database schema statistics

System

Tool	Purpose
`workspace_info`	Current workspace configuration
`switch_workspace`	Switch to a different workspace
`list_workspaces`	List all available workspaces

7. Estimate Tool

Predicts how long a task will take for a specific developer. The most complex MCP tool.

Data Source Priority

#	Source	What it provides
1	`time_spent`	Actual hours logged in Jira
2	Worklogs	Detailed time tracking entries
3	Commit span	First → last commit on the issue
4	`original_estimate`	Manager's estimate
5	Cycle time	Created → resolved (business days)

The tool uses the highest-priority data available and adapts its output accordingly.

With Effort Data (Jira API + Git)

Two predictions per developer:

- **Without bugs** — pure development time
- **With bugs** — real cost including bug aftermath

Based on similar completed tasks, personal speed coefficient from full history, and component familiarity.

With CSV Only (No Effort Data)

When only CSV data is available, the tool shows a **Resolution Timeline**:

- Converts cycle time to **business days** (weekdays only)
- Shows a **range** (min–max across similar tasks)
- Clearly labels it as **lead time** (backlog + dev + review), not active development
- Does **not** apply developer coefficients — multiplying dev speed by backlog wait time is meaningless

Prediction Algorithm

1. **Find similar tasks** — text search + type match + component overlap + recency weighting
2. **Calculate base hours** — weighted trimmed mean of similar tasks
3. **Apply developer coefficient** — personal speed multiplier from history
4. **Apply familiarity factor** — has this dev worked on these components?
5. **Bug overhead** — separate prediction based on developer's historical bug rate

8. Database Schema

PostgreSQL 16 + pgvector, running in Docker on localhost.

Jira Tables

Table	Key Columns
`issues`	issue_key, summary, status, assignee, custom_fields JSONB, raw_json JSONB, embedding vector
`issue_comments`	issue_key, author, body, created
`issue_changelogs`	issue_key, field, from_value, to_value, changed_at
`issue_worklogs`	issue_key, author, time_spent_seconds, started
`issue_links`	source_key, target_key, link_type

Git Tables

Table	Key Columns
`commits`	hash, message, author, committed_at, repo_path, search_vector
`commit_files`	commit_hash, file_path, status, additions, deletions
`commit_issue_refs`	commit_hash, issue_key

GitHub Tables

Table	Key Columns
`pull_requests`	number, title, state, author, merged_at, search_vector
`pr_reviews`	pr_number, reviewer, state
`pr_comments`	pr_number, author, body, path, line
`pr_files`	pr_number, file_path, additions, deletions
`pr_issue_refs`	pr_number, issue_key
`releases`	tag_name, name, body, published_at, author

External Database Tables

Table	Key Columns
`db_tables`	source_name, table_schema, table_name, row_count, size_bytes
`db_columns`	source_name, table_name, column_name, data_type, is_nullable, is_primary_key
`db_foreign_keys`	source_name, table_name, column_name, referenced_table, referenced_column
`db_indexes`	source_name, table_name, index_name, columns, is_unique, is_primary

Design Decisions

- `fields=*all` — every field Jira returns, including all custom fields
- `raw_json` **JSONB** — full original API response, nothing filtered
- **Full-text search** — `tsvector` on issues, commits, PRs
- **pgvector embeddings** — optional 1536-dim vectors for semantic search
- **UPSERT** — idempotent writes, re-syncing is safe
- **Idempotent schema** — `CREATE TABLE IF NOT EXISTS`, runs safely multiple times

9. Testing Strategy

Test Pyramid

Layer	Approach	What it tests
Core types	Unit — pure logic	Config parsing, validation
Adapters (mapper)	Unit — input/output	API response → core types
Adapters (storage)	Unit — mock pg.Pool	SQL generation, UPSERT
Adapters (provider)	Unit — mock API	Pagination, error handling
Use Cases	Integration — fakes	Full flow, in-memory storage
MCP server	MCP transport	Tool registration, req/res
Architecture	Meta-tests	No hardcoded IDs, no eslint-disable

SSOT Fixtures

All test data in one file: `tests/fixtures/shared/test-constants.ts`.

Factory functions: `createIssue()`, `createCommit()`, `createPullRequest()`, `createDbTable()`. When a type changes — update one file, all 631 tests follow.

Quality Gates

- **Pre-commit**: lint-staged → TypeScript check → unit tests
- **CI**: `npm run ci` = typecheck + lint + all tests
- **Architecture tests**: scan for hardcoded IDs, eslint-disable, inline comments

10. Extending Argustack

Adding a New Data Source

Example: adding Linear as an issue tracker.

Step 1 — New adapter:

```
src/adapters/linear/
├── client.ts       API client wrapper
├── mapper.ts       Linear JSON → core Issue type
├── provider.ts     Implements ISourceProvider
└── index.ts
```

Step 2 — Add `'linear'` to `SourceType` union in `core/types/config.ts`.

Step 3 — Add `syncLinear()` in `cli/sync.ts`.

Step 4 — Add Linear setup prompts in `cli/init/setup-linear.ts`.

Nothing else changes. Use cases, storage, MCP tools — all work automatically.

Adding a New MCP Tool

Step 1 — Choose module: `workspace.ts`, `query.ts`, `issue.ts`, `search.ts`, `estimate.ts`, or `database.ts`.

Step 2 — Add row interface in `src/mcp/types.ts` if needed.

Step 3 — Register the tool:

```
server.registerTool('my_tool', {
  description: 'What this tool does',
  inputSchema: {
    param: z.string().describe('Description'),
  },
}, async ({ param }) => {
  const ws = loadWorkspace();
  if (!ws.ok) return errorResponse(ws.reason);
  const { storage } = await createAdapters(ws.root);
  // query, format, return
  return textResponse(result);
});
```

11. Tech Stack

Component	Technology	Role
Language	TypeScript (strict, ESM)	Type safety, async/await
CLI	Commander.js	Driving adapter
MCP	@modelcontextprotocol/sdk	Driving adapter
Jira	jira.js (Version3Client)	Driven adapter
GitHub	Octokit	Driven adapter
Git	es-git (N-API, libgit2)	Driven adapter
CSV	csv-parse	Driven adapter
External DB	Knex (multi-dialect SQL)	Driven adapter
Database	PostgreSQL 16 + pgvector	Storage + vector search
Embeddings	OpenAI text-embedding-3-small	Semantic search
Testing	Vitest	Unit + integration + MCP
Linting	ESLint (strictTypeChecked)	No any, no unsafe ops
Docker	PostgreSQL + pgweb	Infrastructure

Links

Visit the project on GitHub or install from npm.

```
npm i -g argustack
argustack init
argustack sync
```